

NP-Vollständige Probleme

Markus Hecher

Universität Potsdam, Germany

Vorlesung am June 29, 2026

Last updated: 2026-06-29

Zusammenfassung

- Polynomialreduzierbarkeit $\mathcal{P} \leq_p \mathcal{P}'$ bedeutet, es gibt Polynomialzeitreduktion $\mathcal{R}$ von Problem $\mathcal{P}$ auf Problem $\mathcal{P}'$, sodass
 - ↪ I ist positive Instanz von $\mathcal{P} \iff \mathcal{R}(I)$ ist positive Instanz von $\mathcal{P}'$
- Mitglied von C: Obere Schranke, Machbarkeit in C
 - ↪ Reduktion auf Problem in C (oder Algorithmus)
 - Selber fragen: Geht es nicht besser? Z.B.: NP-Mitgliedschaft; geht auch in P?
- Härte bzgl. C: Untere Schranke, Problem ist zmd. so hart wie C
 - ↪ Reduktion von C-vollständigem Problem
 - Selber fragen: Ist das Problem nicht härter? Z.B.: P-Härte; schafft es auch NP?
- Vollständigkeit für C = Mitgliedschaft für C und Härte für C
- Beweise via (1) Aufstellen von $\leq_p$ und (2) Korrektheit beider Richtungen

Inhalt

- 1 NP-Vollständige Probleme
 - Was tun bei NP-Vollständigkeit/-Härte?
 - Ausgewählte Graphen-Probleme
 - Problemlösung durch Reduktion auf SAT
- 2 Komplexität von SAT präzisiert
 - 2-SAT ist in P
 - 3-SAT ist NP-vollständig

NP-Vollständige Probleme

NP-Vollständige Probleme

Was tun bei NP-Vollständigkeit/-Härte?

NP-Vollständigkeit und Techniken für Algorithmen

Wenn ein Problem NP-vollständig (hart) ist, impliziert das, dass das Problem nicht in P ist, außer $NP = P$ (wird als unwahrscheinlich angesehen oder \$\$\$).

Ist ein Problem NP-vollständig (hart), wird oft weiterer Aufwand investiert in:

- Heuristiken
- Spezialfälle
- Parametrisierte Komplexität
- Approximation
- Zufallsalgorithmen (Randomized algorithms)
- (Exponentielle) Algorithmen, die praktisch gut sind für kleine Instanzen
- etc.

NP-Vollständige Probleme

Ausgewählte Graphen-Probleme

Weitere NP-vollständige Graphen-Probleme

Folgende zwei Probleme sind stark miteinander verwandt.

CLIQUE

INSTANZ: Ungerichteter Graph $G = (V, E)$ und $K \in \mathbb{N}$.

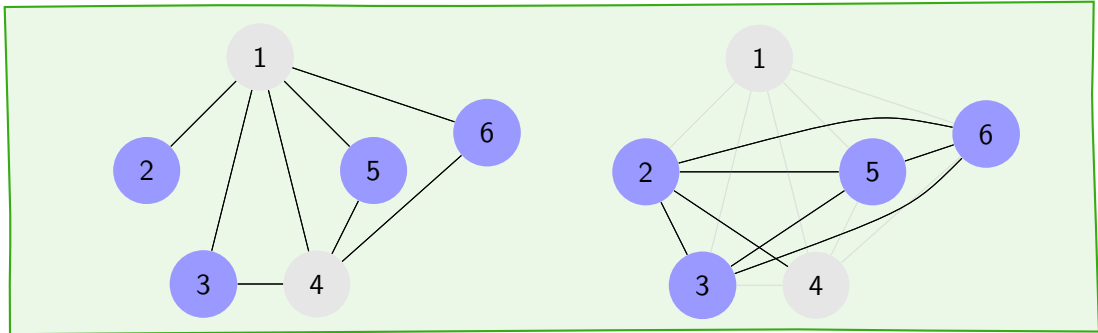
FRAGE: Gibt es eine Clique $C \subseteq V$ der Größe $\geq K$? D.h., existiert $C \subseteq V$, sodass für je zwei $i, j \in C$, $i \neq j$ gilt: $\{i, j\} \in E$?

VERTEX COVER

INSTANZ: Ungerichteter Graph $G = (V, E)$ und $K \in \mathbb{N}$.

FRAGE: Gibt es eine Knotenabdeckung (Vertex Cover) N der Größe $\leq K$? D.h., existiert $N \subseteq V$, sodass für alle $\{i, j\} \in E$ gilt: $i \in N$ oder $j \in N$ (oder beides)?

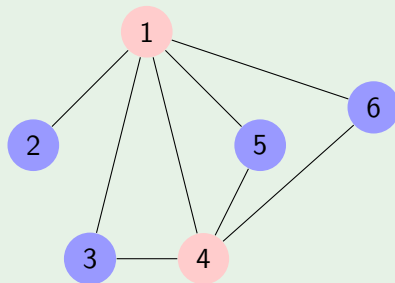
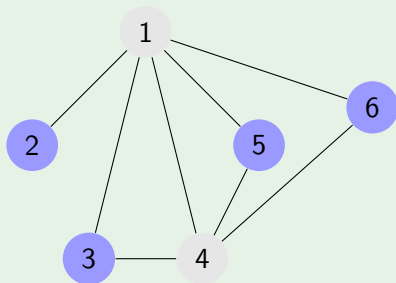
INDEPENDENT SET vs. CLIQUE



Sei $G = (V, E)$ ein ungerichteter Graph und $\overline{G} = (V, \overline{E})$ der **Komplementärgraph**, d.h., $\{i, j\} \in E \Leftrightarrow \{i, j\} \notin \overline{E}$. Dann gilt:
 I ist ein Independent Set in $G \Leftrightarrow I$ ist eine Clique in $\overline{G}$.

INDEPENDENT SET vs. VERTEX COVER

Example 1.



Sei $G = (V, E)$ ein ungerichteter Graph mit $I \subseteq V$. Dann gilt: I ist ein Independent Set in $G \Leftrightarrow N = V \setminus I$ ist ein Vertex Cover in G .

Beweisidee. Ein Independent Set hat nie beide Endpunkte einer Kante. Daher gilt für jede Kante in E , dass zmd. ein Endpunkt in $V \setminus I$ ist.

Komplexität

Theorem 2.

INDEPENDENT SET, **CLIQUE** und **VERTEX COVER** sind NP-vollständig.

Beweis.

Mitgliedschaft. Ein NP-Algorithmus für diese Probleme rät eine Teilmenge S der Kanten V und prüft dann in Polynomialzeit, dass S die gewünschten Eigenschaften hat (z.B., S ist ein Independent Set der Größe $\geq K$).

Härte. Durch die obigen Äquivalenzen (Übung: Independent Set/Vertex Cover), genügt es NP-Härte für eine der Probleme zu zeigen. Wir haben bereits eine Reduktion von **3-SAT** auf **INDEPENDENT SET** gesehen, von der NP-Härte folgt. □

Weitere Graphen-Probleme

k-COLORABILITY (für fixe Konstante $k \geq 1$)

INSTANZ: Ungerichteter Graph $G = (V, E)$

FRAGE: Hat G eine k -Färbung? D.h., gibt es eine Zuweisung jedes Knotens in V zu eine der k Farben, sodass je zwei Knoten i, j , die durch eine Kante verbundenen sind ($\{i, j\} \in E$), nicht die selbe Farbe haben?

3-COLORABILITY ist definiert als **k-COLORABILITY** mit $k = 3$

TSP(E): Problem des Handlungsreisenden

INSTANZ: Städte $1, \dots, n$ und Distanzen $d_{ij} \in \mathbb{N}^+$ zwischen je zwei Städten i und j (sodass $d_{ij} = d_{ji}$) und ein "Budget" B .

FRAGE: Gibt es eine Tour der Länge $\leq B$? D.h., gibt es eine Permutation π , sodass $\sum_{i=1}^n d_{\pi(i)\pi(i+1)} \leq B$ (mit $\pi(n+1) = \pi(1)$)?

Weitere Graphen-Probleme II

HAMILTON-PATH

INSTANZ: (Gerichteter oder ungerichteter) Graph $G = (V, E)$

FRAGE: Hat G einen Hamiltonschen Pfad? D.h., existiert ein Pfad, der alle Knoten von G exakt einmal besucht?

HAMILTON-CYCLE

INSTANZ: (Gerichteter oder ungerichteter) Graph $G = (V, E)$

FRAGE: Hat G einen Hamiltonschen Zyklus? D.h., existiert ein Zyklus, der alle Knoten von G genau einmal besucht?

Weitere Varianten von Erfüllbarkeit

Not-all-equal SAT (**NAESAT**)

INSTANZ: Boolesche Formel φ in 3-CNF

FRAGE: Gibt es eine erfüllbare Belegung T über φ , sodass 3 Literale in jeder Klausel nicht den selben Wahrheitswert haben?

1-IN-3-SAT

INSTANZ: Boolesche Formel φ in 3-CNF

FRAGE: Gibt es eine erfüllbare Belegung T für φ , sodass in jeder Klausel exakt ein Literal zu *true* in T auswertet?

Anmerkung

- Es ist leicht ersichtlich, dass $1\text{-IN-3-SAT} \subset \text{NAESAT} \subset 3\text{-SAT}$ (Probleme gesehen als Menge positiver Instanzen). Die Instanzen dieser 3 Probleme sind ident, nämlich 3-CNF Formeln. Nichtsdestotrotz, die positiven Instanzen von 1-IN-3-SAT sind eine echte Teilmenge jener von NAESAT , die wiederum eine echte Teilmenge der positiven Instanzen von 3-SAT sind.
- ⇒ **Wichtig:** NP-Vollständigkeit irgendeines dieser 3 Probleme impliziert nicht automatisch NP-Vollständigkeit eines anderen Problems, weil a priori nicht klar ist, ob eine weitere Beschränkung der positiven Instanzen das Problem einfacher oder härter macht.
- 3-SAT ist ein Spezialfall von SAT, d.h., die Instanzen von 3-SAT sind eine echte Teilmenge der Instanzen von SAT, also die Frage bleibt exakt ident. Hier liefert NP-Härte vom Spezialfall sofort NP-Härte vom allgemeinen Fall.

NP-Vollständigkeit

Theorem 3.

Folgende Probleme sind NP-vollständig.

- **k-COLORABILITY** für jedes $k \geq 3$ (z.B. **3-COLORABILITY**)
- **HAMILTON-PATH, HAMILTON-CYCLE, TSP(E)**
- **k-SAT** für jedes $k \geq 3$, **NAESAT, 1-IN-3-SAT**

Beweis.

Mitgliedschaft. Alle obigen Problems können gelöst werden, indem man eine Kandidatslösung von polynomieller Größe **rät** und dann in Polynomialzeit **überprüft**, ob die Kandidatslösung tatsächlich eine Lösung ist.

Härte. Konstruktionen kommen teilweise.



NP-Vollständige Probleme

Problemlösung durch Reduktion auf SAT

Motivation

- SAT ist ein kanonisches NP-vollständiges Problem und wird daher oft für NP-Härte anderer Probleme verwendet, indem **von SAT reduziert wird**.
- Allerdings spielen auch **Reduktionen auf SAT** eine wichtige Rolle, weil sich in den letzten Jahrzehnten sehr effiziente “SAT-Solver” entwickelt haben.
- ↪ Eine **Polynomialzeitreduktion** von einem Problem $\mathcal{P} \in \text{NP}$ auf SAT, gemeinsam mit einem SAT-Solver kann ein guter Algorithmus für $\mathcal{P}$ sein.

Reduktion: **3-COLORABILITY** auf **SAT**

Sei $G = (V, E)$ ein Graph (Instanz von **3-COLORABILITY**), sodass $V = \{v_1, \dots, v_n\}$. Wir konstruieren eine **SAT** Formel φ_G wie folgt.

Boolesche Variablen sind von der Form a_i^c , für alle $1 \leq i \leq n$ und $c \in \{r, g, b\}$, um anzugeben, dass Knoten v_i die Farbe c erhält. Wir definieren $\varphi_G = \varphi_1 \wedge \varphi_2 \wedge \varphi_3$:

$$\varphi_1 = \bigwedge_{1 \leq i \leq n} (a_i^r \vee a_i^g \vee a_i^b),$$

$$\varphi_2 = \bigwedge_{1 \leq i \leq n} (\neg a_i^r \vee \neg a_i^g) \wedge (\neg a_i^r \vee \neg a_i^b) \wedge (\neg a_i^g \vee \neg a_i^b),$$

$$\varphi_3 = \bigwedge_{\{v_i, v_j\} \in E} (\neg a_i^r \vee \neg a_j^r) \wedge (\neg a_i^g \vee \neg a_j^g) \wedge (\neg a_i^b \vee \neg a_j^b).$$

Es verbleibt zu zeigen: G ist 3-färbbar $\Leftrightarrow \varphi_G$ ist erfüllbar.

Reduktion: **3-COLORABILITY** auf **SAT**

Um “ $\Rightarrow$ ” zu zeigen, sei $G = (V, E)$ 3-färbbar. Wir müssen zeigen, dass φ_G erfüllbar ist.

Sei $\mu : V \rightarrow \{r, g, b\}$ eine Farbbelegung, die 3-Färbbarkeit von G nachweist. Um Erfüllbarkeit von φ_G zu zeigen, definieren wir eine Belegung T , die φ_G zu *true* auswertet. Wir definieren T wie folgt:

$$T(a_i^c) = \begin{cases} \text{true} & \text{wenn } \mu(v_i) = c, \\ \text{false} & \text{wenn } \mu(v_i) \neq c. \end{cases}$$

Um zu zeigen, dass φ_G unter T zu *true* auswertet, genügt es, zu verifizieren, dass alle $\varphi_1, \varphi_2, \varphi_3$ zu *true* auswerten.

- μ färbt jeden Knoten von G mit zumindest einer Farbe. Durch die Konstruktion von T , wertet φ_1 unter T zu *true* aus.
- μ färbt einen Knoten von G nie mit mehr als einer Farbe. Durch die Konstruktion von T , wertet daher auch φ_2 unter T zu *true* aus.

Reduktion: **3-COLORABILITY** to **SAT** (ctd.)

- Per Definition: $\varphi_3 = \bigwedge_{\{v_i, v_j\} \in E} (\neg(a_i^r \wedge a_j^r) \wedge \neg(a_i^g \wedge a_j^g) \wedge \neg(a_i^b \wedge a_j^b))$.
 - Wir wissen, dass μ eine 3-Färbung von G ist, d.h., für jede $\{v_i, v_j\} \in E$, gilt (*) $\mu(v_i) \neq \mu(v_j)$.
 - Widerspruchsbeweis: Angenommen, φ_3 würde unter T zu *false* auswerten. Dann muss es $\{v_i, v_j\} \in E$ und $c \in \{r, g, b\}$, sodass Teilformel $\neg(a_i^c \wedge a_j^c)$ unter T zu *false* auswertet.
Daher wertet $(a_i^c \wedge a_j^c)$ unter T zu *true* aus. Es folgt dann, dass $T(a_i^c) = \text{true}$ und $T(a_j^c) = \text{true}$. Durch die Konstruktion von T folgt dann $\mu(v_i) = \mu(v_j) = c$. Das widerspricht (*), und daher wertet φ_3 unter T zu *true* aus.

Reduktion: **3-COLORABILITY** auf **SAT** (fortgesetzt)

Um " $\Leftarrow$ " zu beweisen, nehmen wir an, dass φ_G eines Graphen $G = (V, E)$ erfüllbar sei. Wir müssen zeigen, dass der Graph G dann 3-färbbar ist.

Dadurch, dass φ_G erfüllbar ist, gibt es eine Belegung T unter der φ_G zu *true* ausgewertet. Sei $1 \leq i \leq n$ beliebig. T wertet φ_1 zu *true* aus, also ist zumindest eines der Atome a_i^r, a_i^g und a_i^b unter T auch *true*. T wertet auch φ_2 zu *true* aus, d.h., maximal ein Atom unter a_i^r, a_i^g und a_i^b ist *true* unter T .

Wir definieren die Funktion $\mu : V \rightarrow \{r, g, b\}$, sodass für alle $1 \leq i \leq n$, $\mu(v_i) = c$ mit $c \in \{r, g, b\}$ der eindeutige Wert ist für den gilt $T(a_i^c) = \text{true}$.

Es verbleibt zu zeigen, dass μ eine 3-Färbung ist. Angenommen, es gäbe eine Kante $\{v_i, v_j\} \in E$, sodass $\mu(v_i) = \mu(v_j) = c$ für $c \in \{r, g, b\}$. Per Konstruktion von μ , sind a_i^c und a_j^c auf *true* gesetzt in T .

Es gilt: $\varphi_3 = \bigwedge_{\{v_i, v_j\} \in E} (\neg(a_i^r \wedge a_j^r) \wedge \neg(a_i^g \wedge a_j^g) \wedge \neg(a_i^b \wedge a_j^b))$. Dann wertet $\neg(a_i^c \wedge a_j^c)$ in φ_3 unter T zu *false* aus. $\rightsquigarrow$ Widerspruch: φ_3 wertet laut Annahme zu *true* aus.

Graphen-Homomorphismus

HOMOMORPHISMUS

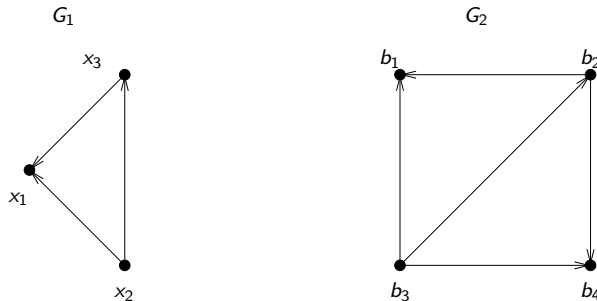
INSTANZ: Zwei gerichtete Graphen $G_1 = (V_1, E_1)$ und $G_2 = (V_2, E_2)$

FRAGE: Gibt es einen Homomorphismus von G_1 auf G_2 ? D.h., gibt es eine Abbildung $h : V_1 \rightarrow V_2$, sodass gilt: Wenn $(v_1, v_2) \in E_1$, dann $(h(v_1), h(v_2)) \in E_2$?

HOMOMORPHISMUS ist einer der Grundaufgaben bei Datenbankabfragen:

- G_1 kann man als Abfrage (query) sehen und G_2 als Datenbank.
- Wir fragen also, ob G_1 eine positive Abfrage der Datenbank G_2 ist.
- **SQL-Abfragen** der Form **select-from-where** sind im Wesentlichen versteckte **HOMOMORPHISMUS**-Fragen.

Graphen-Homomorphismus: Ein Beispiel (1)



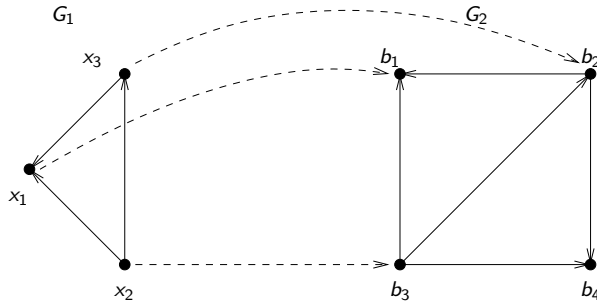
G_2 kann man als Datenbank mit einer einzigen Relation $E(start, end)$ sehen:

$$E = \{(b_3, b_1), (b_2, b_1), (b_3, b_2), (b_2, b_4), (b_3, b_4)\}.$$

Die Berechnung aller Homomorphismen von G_1 auf G_2 entspricht der Beantwortung der folgenden SQL-Abfrage

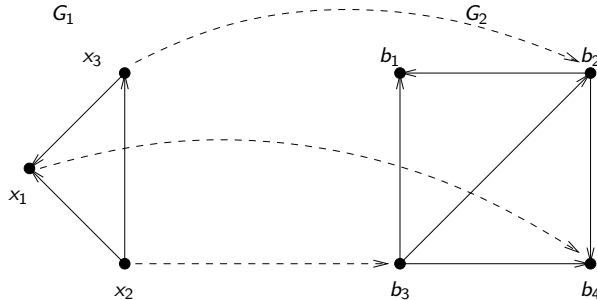
```
select e1.end, e1.start, e2.start      from E e1, E e2, E e3      where e1.start =
e3.start and e1.end = e2.end and e2.start = e3.end
```


Graphen-Homomorphism: Ein Beispiel (2)



Der obige Homomorphismus zeigt, dass (b_1, b_3, b_2) in der Antwort von Q ist.

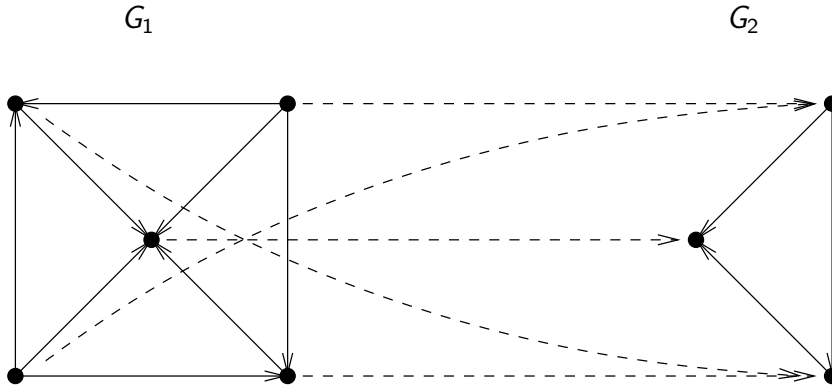
Graph Homomorphism: An Example (3)



Der obige Homomorphismus zeigt, dass (b_4, b_3, b_2) in der Antwort von Q ist.

Gibt es weitere Homomorphismen (und Antworten auf Q)? Nein.

Graphen-Homomorphismen: Ein größeres Beispiel



Reduktion von Homomorphismus auf SAT

Reduktion

Sei eine beliebige Instanz von HOMOMORPHISMUS gegeben durch zwei gerichtete Graphen $G_1=(V_1, E_1)$ und $G_2=(V_2, E_2)$. Sei $V_1=\{a_1, \dots, a_n\}$ und $V_2=\{b_1, \dots, b_m\}$.

Wir müssen eine Formel φ definieren (Übung), sodass φ erfüllbar $\iff$ es gibt Homomorphismus from G_1 to G_2 .

Die Formel φ verwendet die folgenden **Booleschen Variablen** (Vorschlag):

- M_{a_i, b_j} für jedes $1 \leq i \leq n$ und $1 \leq j \leq m$ (bedeutet: wenn M_{a_i, b_j} auf *true* gesetzt wird, dann bildet a_i von G_1 auf b_j von G_2 ab)
- E_{a_i, a_j}^1 für alle $1 \leq i, j \leq n$ (E_{a_i, a_j}^1 bedeutet G_1 hat eine Kante von a_i auf a_j)
- E_{b_i, b_j}^2 für alle $1 \leq i, j \leq m$ (E_{b_i, b_j}^2 bedeutet G_2 hat eine Kante von b_i auf b_j)

Komplexität von SAT präzisiert

Varianten von Erfüllbarkeit

SAT

INSTANZ: Boolesche Formel φ .

FRAGE: Ist φ erfüllbar?

3-SAT

INSTANZ: Boolesche Formel φ in 3-CNF

FRAGE: Ist φ erfüllbar?

2-SAT

INSTANZ: Boolesche Formel φ in 2-CNF

FRAGE: Ist φ erfüllbar?

Komplexität von SAT präzisiert

2-SAT ist in P

Reduktion von 2-SAT auf Reachability

Es gibt eine Polynomialzeit-Reduktion von dem **2-SAT**-Problem auf das **REACHABILITY**-Problem.

Anmerkung

Wir haben bereits gesehen, dass das **REACHABILITY** Problem handhabbar ist (d.h., es kann in Polynomialzeit gelöst werden).

Folgerung: Das **2-SAT**-Problem ist handhabbar.

Reduktion von 2-SAT auf Reachability II

Beweis (Skizze).

Sei φ eine beliebige Instanz von **2-SAT**.

Wir definieren einen Graphen $G(\varphi)$ wie folgt:

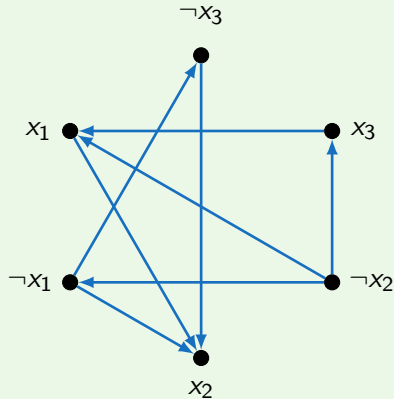
- Die Variablen von φ und deren Negationen formen die Knoten von $G(\varphi)$.
- Es gibt eine (gerichtete!) Kante (α, β) gdw. es eine Klausel $\bar{\alpha} \vee \beta$ oder $\beta \vee \bar{\alpha}$ in φ gibt, d.h., wenn (α, β) eine Kante ist, dann ist $(\bar{\beta}, \bar{\alpha})$ auch eine Kante ($\bar{\alpha}$ entspricht dem Komplement von α).
- **Semantik** der Kanten (α, β) : Wenn α *true* ist in einer erfüllbaren Belegung T von φ , dann muss β auch *true* sein in T .

Man kann zeigen, dass φ unerfüllbar ist, gdw. es eine Variable x und Pfade in $G(\varphi)$ gibt, nämlich von x nach $\neg x$ und von $\neg x$ nach x . □

Beispiel.

$$\psi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

$\neg x_1 \Rightarrow x_2$
 $\neg x_2 \Rightarrow x_1$
 $\neg x_1 \Rightarrow \neg x_3$
 $x_3 \Rightarrow x_1$
 $x_1 \Rightarrow x_2$
 $\neg x_2 \Rightarrow \neg x_1$
 $\neg x_2 \Rightarrow x_3$
 $\neg x_3 \Rightarrow x_2$



Beweis (Skizze) fortgesetzt.

Wir erhalten damit folgenden Algorithmus für **2-SAT**:

- 1 Konstruiere den Graphen $G(\varphi)$ wie oben beschrieben;
- 2 Für alle Variablen x von φ : Prüfe, ob sowohl $\neg x$ von x im Graphen $G(\varphi)$ und x von $\neg x$ in $G(\varphi)$ erreichbar sind;
- 3 Wenn durch die REACHABILITY-Tests in Schritt 2, eine Variable x gefunden wird, sodass sowohl $\neg x$ von x als auch x von $\neg x$ erreicht werden, dann gibt der **2-SAT**-Algorithmus ein "Nein" zurück. Andernfalls returnieren er "Ja".

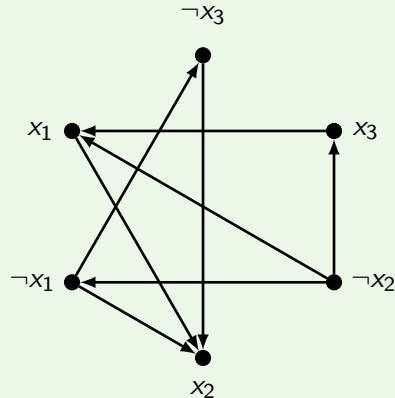


Beispiel (fortgesetzt).

$$\psi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

- x_1 erreicht nur x_2
- x_2 erreicht keinen anderen Knoten
- x_3 erreicht nur x_1 und x_2

~> ψ ist erfüllbar



Komplexität von SAT präzisiert

3-SAT ist NP-vollständig

Varianten von Erfüllbarkeit

- Wir wissen, dass **2-SAT** **tractable** ist:
Man kann einen Polynomialzeit-Algorithmus zum Beispiel auch über eine Polynomialzeit-Reduktion auf **REACHABILITY** erhalten.
- Im Gegenzug sind SAT und **3-SAT** **nicht handhabbar**.

Das SAT-Problem ist DAS klassische NP-vollständige Problem. Es verbleibt NP-vollständig, selbst wenn wir die Boolesche Formel auf 3-CNF beschränken (**3-SAT**).

Komplexität von SAT und 3-SAT

Satz von Cook-Levin: SAT ist NP-vollständig.

Es gilt sogar die folgende stärkere Variante.

3-SAT ist NP-vollständig.

Beweis der NP-Mitgliedschaft.

SAT und auch **3-SAT** kann man durch den folgenden NP-Algorithmus nachweisen:

1. Rate eine Belegung T über die Variablen in φ .
2. Überprüfe, dass φ in T zu *true* auswertet.



Beweisidee der NP-Härte von SAT.

Um zu zeigen, dass SAT NP-hart ist, müssen wir für **jedes** Problem $\mathcal{P}$ in NP zeigen, dass es eine Polynomialzeit-Reduktion von $\mathcal{P}$ auf SAT gibt.

Dadurch, dass NP unendlich viele Probleme hat, können wir nicht jedes Problem separat behandeln. Wir brauchen eine Methode, um alle Probleme in NP gemeinsam zu adressieren.

Die Idee ist wie folgt. Wir nehmen ein **beliebiges Problem** $\mathcal{P} \in \text{NP}$ und eine **beliebige Instanz** I von $\mathcal{P}$. Wir müssen zeigen, dass wir in Polynomialzeit eine Formel φ bauen können, sodass **I eine positive instanz von $\mathcal{P}$ ist $\Leftrightarrow \varphi$ ist erfüllbar**.

Weil $\mathcal{P} \in \text{NP}$, muss es per Definition eine polynomiell-balancierte und polynomiell-entscheidbare Zertifikatsrelation R für $\mathcal{P}$ geben.

Per Definition: I ist eine positive instanz von $\mathcal{P} \Leftrightarrow$ es gibt ein Objekt C von polynomieller Größe in $|I|$, sodass $(I, C) \in R$. Um $(I, C) \in R$ zu testen, haben wir einen Polynomialzeit-Algorithmus A zur Verfügung.

Beweisidee der NP-Härte von SAT II.

Für die Reduktion genügt es daher, in Polynomialzeit eine Formel φ zu konstruieren, sodass folgende Äquivalenz gilt:

φ ist erfüllbar $\Leftrightarrow$ es gibt ein polynomiell großes Objekt C , sodass Algorithmus A auf Eingabe (I, C) den Wert *true* liefert.

Die **gewünschte Boolesche Formel** φ muss 2 Aufgaben ausführen:

- 1 Die Kandidatszertifikate genrieren,
- 2 Die Berechnung von A auf einem Kandidatspaar (I, C) simulieren.

Die 2. Aufgabe ist nicht trivial: Wie können wir eine Boolesche Formel verwenden, um Pseudocode zu simulieren? Wie behandeln wir **while**, **for**, **if/then/else**?

Antwort: Das kann man prinzipiell machen, aber es ist viel einfacher, stattdessen einfach **Turingmaschinen** zu verwenden (und die Church-Turing These). Turingmaschinen sind ein **viel einfacheres Berechnungsmodell**, das aber immer noch stark genug ist im Hinblick der Berechenbarkeit und Komplexität. □

Komplexität von SAT und 3-SAT II

Beweisidee der NP-Härte von **3-SAT**.

Wir müssen von SAT auf **3-SAT** reduzieren, d.h.: Sei φ eine beliebige Boolesche Formel. We zeigen, dass es eine Boolesche Formel $R(\varphi) = \psi$ gibt, sodass ψ in 3-CNF ist und φ erfüllbar ist $\Leftrightarrow \psi$ erfüllbar ist.

Anmerkung.

- Eine beliebige Boolesche Formel φ kann in **Polynomialzeit** in eine **erfüllungs-äquivalente** Formel ψ in 3-CNF übersetzt werden.
- Im Allgemeinen sind φ und ψ **nicht logisch äquivalent** (wegen Hilfsvariablen!).
- Dieses Resultat ist nicht trivial: Die “einfache” Übersetzung nach CNF via den Gesetzen von de Morgan's laws und Distributivität von $\wedge$ und $\vee$ führt üblicherweise zu einem exponentiellen Blow-up.

Beispiel: Die Formel, die **logisch äquivalent** zu $(x_1 \wedge y_1) \vee \dots \vee (x_n \wedge y_n)$ ist, ist exponentiell größer.

Zusammenfassung

- Definitionen von C-harte und C-vollständige Probleme für eine Klasse C.
- Verständnis für die Rolle vollständiger Probleme in der Komplexitätstheorie
- Sie sollten die Intuition von NP-Vollständigkeit verstanden haben (und NP-vollständige Probleme erkennen).
- Zwei fundamentale NP-vollständige Probleme: SAT und **3-SAT**.
- Viele Beispiele NP-vollständiger Probleme, z.B.: **CLIQUE**, **INDEPENDENT SET**, **VERTEX COVER**, **3-COLORABILITY**, **HAMILTON-PATH**, **HAMILTON-CYCLE**, **TSP(E)**, etc.